

Section - A

Ans 1. CPU registers are small, high-speed storage locations inside the processors.

Importance:

- Store intermediate data and instructions.
- Faster than cache and memory.
- Improve execution speed of programs.

Ans 2. Program counter (PC) is a register that holds the address of the instruction.

Function:

- Points to next instruction to be executed.
- Automatically update after each instruction.
- Controls instruction sequence.

Ans 3. Instruction format defines the structure of an instruction in binary form.

Key fields:

- Opcode (operation)
- Operand
- Addressing mode

Ans 4. Addressing mode specify how to locate operands.

Purpose:

- Provide flexibility in accessing data.
- Reduce instruction count.
- Improve program efficiency.

Ans 6: ISA is the interface between hardware and software defining supported instructions.

Includes:

- Instruction formats
- Data types
- Registers and addressing modes.

Ans 7: Types of Instructions in a processor are:

- Arithmetic (ADD, SUB)
- Logical (AND, OR)
- Data Transfer (LOAD, STORE)
- Control (JUMP, BRANCH)

Ans 8: Explanation:

- opcode is interpreted
- Required operation is identified
- control signals are generated
- operands are prepared long execution.

Ans 9: Basic instruction cycle are:

- fetch instruction from memory
- decode instruction
- execute operation
- store result.

Ans 10: Characteristics of RISC Processors are:

- simple and fixed length instructions.
- few addressing modes
- faster execution (Pipeline-friendly)
- Load/store architecture.

11. characteristics of CUC Processors are:
- complex and variable-length instructions.
 - many addressing modes.
 - fewer instructions per program
 - slower but powerful instructions.

Ans 12: RISC-V:

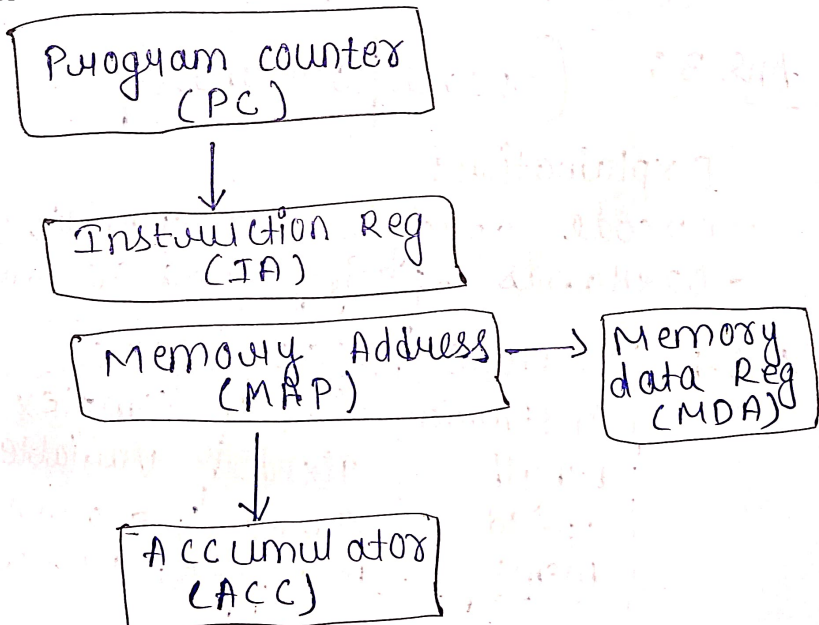
- open-source ISA
- simple, fixed-length instructions
- easier to design and extend.

x80:

- Proprietary (Intel/AMD)
- complex, variable-length instructions.

Section-B

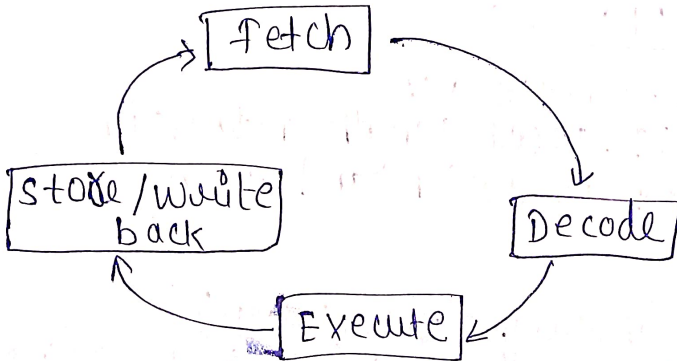
Ans 1:



Explanation:

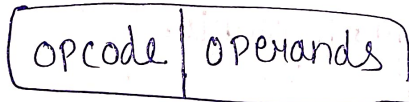
- PC holds next instruction address; IR stores current instruction.
- MAR & MDR handle memory access; Acc. stores intermediate results.

Ans 2:



- CPU fetches instructions, decodes it and executes it and writes back.

Ans 3:



Explanation:

- opcode specifies the operations
- operands specify data or memory location.

Ans 4:

Feature	RISC	CISC
Instruction	Simple	Complex
Length	fixed	variable
speed	fast	slower
memory	more usage	less usage

Explanation:

- RISC focuses on simplicity and speed.
- CISC focuses on powerful and complexity

Section - C

Ans 1: Impact on performance are:
→ memory access is slower than register access.
→ Increases CPI due to memory latency
→ overall execution time increases → performance decrease.

Ans 2: One should choose RISC architecture because:

- simple and fixed-length instructions
- easier to pipeline efficiently
- Requires simple hardware design.

Ans 3: CPU executes this instruction as:

Step 1: Fetch

- Instruction is fetched from memory into IR.

Step 2: Decode

- Opcode is decoded and registers are identified.

Step 3: Execute

- Data is read from registers
- ALU performs operation.
- Result is stored back in register

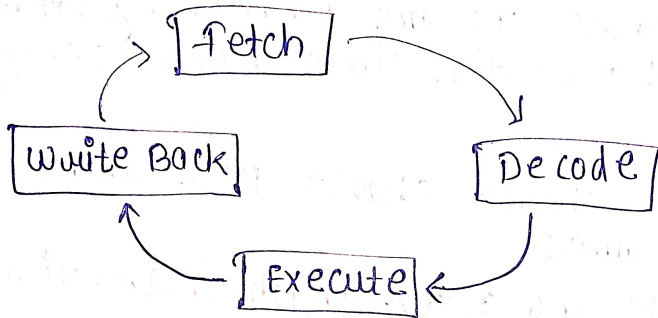
Ans 4: x86 is widely used despite complexity because:

- Backward compatibility with older software.
- Large ecosystem (OS, applications, industry, support).

- Modern CPUs internally optimized.
- Despite complexity, compatibility and ecosystem keep x86 dominant.

Section-D

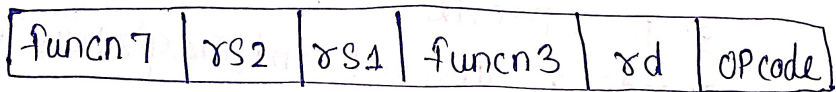
Ans 1:



Explanation of Fetch stage:

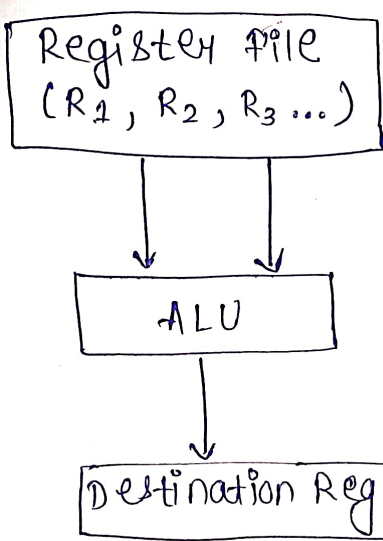
- PC holds address of next instructions.
- Address sent to memory via MAR.
- Instruction fetched into MDR and loaded into IR.
- PC is incremented to next instruction

Ans 2:



Explanation:

- Opcode - specifies operation type.
- rd - Destination register
- funcn 3, rs1, rs2 - Source register
- funcn 3 & funcn 7 - Define exact operation
- Fixed-Length (32-bit) format simplifies doing.



Explanation:

- Data is read from source registers.
- sent to ALU for operation
- Result is written back to destination register.
- Enable fast computation without memory access.